

Práctica 5 - Capa de Transporte - Parte 1

1. ¿Cuál es la función de la capa de transporte?

La capa de transporte tiene como función principal proveer comunicación lógica extremo a extremo entre procesos que se ejecutan en dispositivos distintos dentro de la red.

Esto significa que:

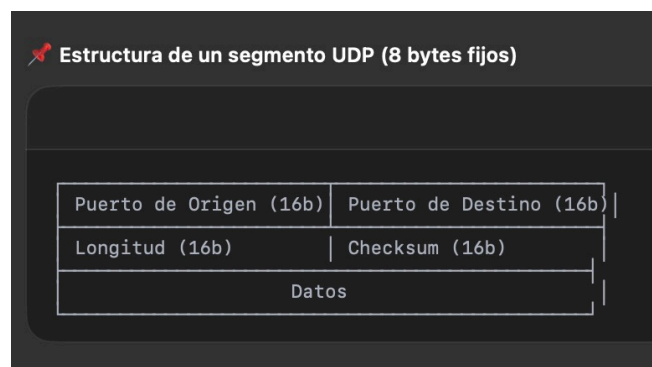
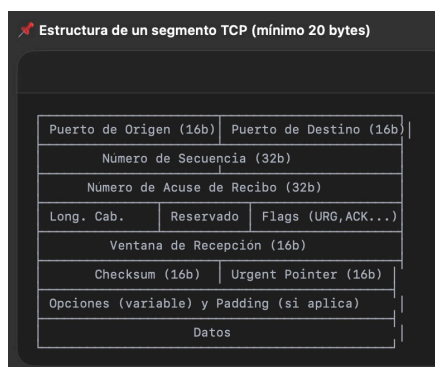
- Se encarga de **segmentar** los datos de la aplicación en unidades manejables (segmentos) y reensamblarlos en el destino.
- **Identifica a cada aplicación** usando números de puerto, para que varias aplicaciones puedan usar la red simultáneamente.
- **Entrega confiable** (*si el protocolo lo soporta*): detecta errores, pérdidas y desorden de segmentos y los corrige (por ejemplo, TCP).
- **Controla el flujo**: regula la velocidad de envío para no saturar al receptor.
- Puede **implementar control de congestión**: ajusta la tasa de envío para evitar sobrecargar la red.

En resumen, **la capa de transporte actúa como un intermediario inteligente entre la capa de aplicación y la de red**, asegurando que los datos lleguen de manera correcta, ordenada y eficiente al proceso destino.

Función	Descripción
Comunicación lógica extremo a extremo	Permite que dos procesos en hosts distintos se comuniquen como si tuvieran un enlace directo entre ellos.
Segmentación y reensamblado	Divide los datos de la aplicación en segmentos para su envío y los reconstruye en el destino en el orden correcto.
Multiplexación y demultiplexación	Usa puertos para identificar aplicaciones de origen y destino, permitiendo que múltiples procesos compartan la red simultáneamente.
Entrega confiable (si aplica)	Con protocolos como TCP asegura que los datos lleguen sin errores, sin pérdidas y en orden.
Control de flujo	Ajusta la velocidad de envío para no sobrecargar al receptor.
Control de congestión	Regula la tasa de transmisión para evitar saturar la red y mantener un uso eficiente de los recursos.

2. Describa la estructura del segmento TCP y UDP.

- **Cabecera TCP**: más larga (mínimo 20 bytes, puede crecer con opciones), incluye mecanismos de fiabilidad, control de flujo y control de congestión.
- **Cabecera UDP**: muy corta (8 bytes fijos), no incluye control de flujo ni reordenamiento, es más rápida y simple.



Diferencia visual clave: la cabecera TCP es más larga y tiene muchos campos de control (secuencia, ACK, flags, ventana), mientras que la cabecera UDP es mínima (solo puertos, longitud y checksum).

Estructura de los Segmentos TCP y UDP

Campo	TCP	UDP
Puerto de origen (16 bits)	✓	✓
Puerto de destino (16 bits)	✓	✓
Longitud total	Implícita en la longitud del datagrama IP, no está en el header.	✓ (16 bits, incluye cabecera + datos)
Checksum (16 bits)	✓ (obligatorio en IPv6, opcional en IPv4)	✓ (obligatorio)
Número de secuencia (32 bits)	✓ – Identifica el primer byte de datos en el segmento, usado para orden y control de duplicados.	✗
Número de acuse de recibo (ACK) (32 bits)	✓ – Indica el próximo byte que se espera recibir.	✗
Longitud de cabecera / Offset de datos (4 bits)	✓ – Indica dónde comienzan los datos.	✗
Flags de control (6 bits)	✓ – URG, ACK, PSH, RST, SYN, FIN. Controlan el establecimiento de conexión y transmisión.	✗
Ventana de recepción (16 bits)	✓ – Implementa control de flujo, indica cuántos bytes puede recibir el host.	✗
Urgent Pointer (16 bits)	✓ – Apunta al último byte de datos urgentes (cuando URG está activo).	✗
Opciones	✓ – Por ejemplo, MSS, escalado de ventana, timestamps.	✗
Datos (Payload)	✓ – Tamaño variable.	✓ – Tamaño variable.

3. ¿Cuál es el objetivo del uso de puertos en el modelo TCP/IP?

El objetivo del uso de puertos en el modelo TCP/IP es permitir la **multiplexación y demultiplexación de datos** entre múltiples aplicaciones que usan simultáneamente la red en un mismo host.

En otras palabras:

- Un **número de puerto** identifica a la aplicación o proceso dentro de un dispositivo.
- Gracias a los **puertos**, un host puede tener **varias conexiones abiertas al mismo tiempo** (por ejemplo, un navegador web en el puerto 443, un cliente de correo en el puerto 993 y un servicio SSH en el puerto 22).
- La capa de transporte (TCP o UDP) usa el par (**puerto origen, puerto destino**) para entregar cada segmento al proceso correcto.

Ejemplo:

*Si escribís www.unlp.edu.ar en el navegador, tu cliente **TCP** usa un puerto de origen dinámico (por ejemplo, 52344) y conecta con el puerto de destino 80 (HTTP) o 443 (HTTPS) en el servidor. Cuando llegan las respuestas, la capa de transporte sabe que debe entregarlas al proceso que abrió el puerto 52344 en tu computadora (tu navegador).*

Función	Descripción / Ejemplo
Identificar aplicaciones	Cada puerto corresponde a un proceso o aplicación en el host.
Multiplexación	Permite que múltiples aplicaciones compartan la misma conexión de red. Ej: navegador, cliente de correo y SSH funcionando a la vez.
Demultiplexación	La capa de transporte entrega cada segmento al proceso correcto usando el par (puerto origen, puerto destino).
Ejemplo práctico	Cliente: puerto dinámico 52344 → Servidor: puerto 443 (HTTPS). Cuando llega la respuesta, TCP sabe que debe entregarla al navegador que abrió el puerto 52344.

4. Compare TCP y UDP en cuanto a:

- Confiability
- Multiplexación.
- Orientado a la conexión.
- Controles de congestión.
- Utilización de puertos.

Aspecto	TCP	UDP
Confiabilidad	✓ Proporciona entrega confiable : confirma recepción (ACK), retransmite en caso de pérdida, asegura orden de los segmentos.	✗ No confiable : no retransmite ni garantiza orden. La aplicación debe manejar pérdidas si lo necesita.
Multiplexación / Demultiplexación	✓ Usa puertos de origen y destino para identificar aplicaciones.	✓ También usa puertos de origen y destino. (Ambos protocolos permiten multiplexación/demultiplexación).
Orientado a la conexión	✓ Sí. Requiere establecer una conexión previa (three-way handshake) antes de transferir datos.	✗ No. Es no orientado a conexión : envía datagramas sin handshake previo.
Control de congestión	✓ Implementa algoritmos de control de congestión (AIMD, slow start, etc.) para evitar saturar la red.	✗ No implementa control de congestión: envía a la máxima velocidad posible.
Utilización de puertos	✓ Usa puertos bien conocidos y efímeros, igual que UDP. (No hay diferencia en el mecanismo de puertos).	✓ Idem TCP.

Resumen:

- **TCP**: más pesado, confiable, orientado a conexión, con control de flujo y congestión.
- **UDP**: liviano, sin conexión, sin confiabilidad ni control de congestión, ideal para aplicaciones en tiempo real (voz, video, DNS).

5. La PDU de la capa de transporte es el segmento. Sin embargo, en algunos contextos suele utilizarse el término datagrama. Indique cuando.

La **PDU** (Unidad de Datos de Protocolo) de la capa de transporte se llama **segmento** cuando usamos **TCP**.

Sin embargo, en el caso de UDP, por su simplicidad y porque no está orientado a conexión, la PDU recibe el nombre de datagrama UDP.

Resumen:

- **TCP** → Segmento
- **UDP** → Datagrama

El término **datagrama** también se usa para remarcar que la entrega es independiente, sin garantías de orden ni confiabilidad (similar al datagrama IP en la capa de red).

6. Describa el saludo de tres vías de TCP . ¿UDP tiene esta característica?

El saludo de tres vías es el proceso que usa **TCP** para establecer una conexión confiable entre cliente y servidor antes de transferir datos.

Pasos:

1. **SYN**: el cliente envía un segmento con el flag **SYN** en 1 y su número de secuencia inicial.
2. **SYN-ACK**: el servidor responde con un segmento que tiene **SYN=1** y **ACK=1**, confirmando la recepción del **SYN** y enviando su propio número de secuencia inicial.
3. **ACK**: el cliente envía un último segmento con **ACK=1** confirmando la recepción del **SYN** del servidor.

Recién después de estos tres pasos, la conexión se considera establecida y se pueden enviar datos.

Propósito:

- Sincroniza números de secuencia en ambos sentidos.
- Confirma que ambas partes están listas para comunicarse.
- Crea un estado de conexión en cliente y servidor.

UDP no tiene saludo de tres vías.

Es un protocolo **no orientado a conexión**, por lo que simplemente envía datagramas sin establecer previamente un canal de comunicación.

7. Investigue qué es el ISN (Initial Sequence Number). Relaciónelo con el saludo de tres vías.

El **ISN** es el número de secuencia inicial que cada extremo (cliente y servidor) elige al iniciar una conexión TCP.

- Es un valor de 32 bits seleccionado de manera pseudoaleatoria para cada nueva conexión.
- Sirve como punto de partida para numerar los bytes de datos que se enviarán en esa sesión.
- Su uso evita que segmentos viejos de conexiones anteriores sean aceptados por error en una conexión nueva (protección contra replay).

Relación con el saludo de tres vías

Durante el saludo de tres vías, tanto cliente como servidor intercambian sus ISN:

- **SYN**: el cliente envía su ISN al servidor.
- **SYN-ACK**: el servidor responde enviando su propio ISN y confirmando el del cliente con un ACK.
- **ACK final**: el cliente confirma el ISN del servidor.

De esta manera, al terminar el saludo de las tres vías, ambos extremos saben desde qué número de secuencia deben empezar a contar y pueden reconocer cada byte de datos transmitido en la conexión.

8. Investigue qué es el MSS. ¿Cuándo y cómo se negocia?

El **MSS** es el tamaño máximo de datos útiles (payload) que un host está dispuesto a recibir en un único segmento TCP.

- No incluye la cabecera TCP ni la IP, solo la carga de aplicación.
- Busca evitar la fragmentación a nivel IP, eligiendo un tamaño que quepa en una sola trama de la red subyacente.
- Su valor típico en redes Ethernet es de **1460 bytes** (1500 bytes de MTU – 20 bytes de cabecera IP – 20 bytes de cabecera TCP).

Cuándo y cómo se negocia

El MSS se **negocia** durante el saludo de tres vías de TCP:

- Cada extremo envía su MSS como **opción TCP** en el segmento **SYN**.
- El valor que se usa finalmente para la conexión es el **mínimo** de los dos MSS anunciados (para que ambos lados estén de acuerdo y evitar que el receptor reciba segmentos demasiado grandes).

Importancia

- Evita fragmentación IP (que reduce rendimiento y puede generar pérdidas si un fragmento se pierde).
- Optimiza el rendimiento de la transmisión adaptando el tamaño de los segmentos a la red real.

Ejemplo de esquema sencillo de la negociación de MSS en el saludo de tres vías



9. Utilice el comando ss (reemplazo de netstat) para obtener la siguiente información de su PC:

✓ Resumen rápido de flags:

- -t → TCP
- -u → UDP
- -l → solo sockets en escucha (listening)
- -p → mostrar el proceso asociado

a. Para listar las comunicaciones TCP establecidas.

```
redes@debian:~$ ss -t state established
```

Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
0	0	192.168.64.2:bootpc	192.168.64.1:bootps	

El comando muestra que no hay conexiones TCP establecidas en el sistema.

Solo aparecen los encabezados de la tabla, lo que indica que ningún proceso está usando TCP activamente en ese momento.

b. Para listar las comunicaciones UDP establecidas.

```
redes@debian:~$ ss -u state established
```

Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
0	0	192.168.64.2:bootpc	192.168.64.1:bootps	

El comando muestra una comunicación **UDP activa** entre la dirección local 192.168.64.2 (puerto bootpc) y el servidor 192.168.64.1 (puerto bootps).

Esto corresponde al **protocolo DHCP**, usado para asignar automáticamente una dirección IP a la máquina. A diferencia de TCP, UDP no establece una conexión formal, por eso solo se muestra el intercambio entre cliente y servidor sin estado de conexión persistente.

c. Obtener sólo los servicios TCP que están esperando comunicaciones.

```
redes@debian:~$ ss -lt
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	128	0.0.0.0:ssh	0.0.0.0:*	
LISTEN	0	128	127.0.0.1:ipp	0.0.0.0:*	
LISTEN	0	5	127.0.0.1:4038	0.0.0.0:*	
LISTEN	0	128	:::ssh	:::*	
LISTEN	0	128	:::ipp	:::*	
LISTEN	0	4096	:::1:50051	:::*	
LISTEN	0	4096	:::ffff:127.0.0.1:50051	:::*	

El comando muestra los **servicios TCP que están en modo de escucha (LISTEN)**, es decir, esperando conexiones entrantes.

En la salida se observan varios puertos abiertos, como:

- ssh (puerto 22) → permite conexiones remotas seguras al sistema.
- ipp (puerto 631) → usado por el servicio de impresión (Internet Printing Protocol).
- Otros puertos locales (4038, 50051) → pertenecen a servicios internos o de pruebas en la máquina.

Las direcciones 0.0.0.0 y ::: indican que los servicios escuchan en todas las interfaces de red (IPv4 e IPv6).

En resumen, el sistema **tiene varios procesos TCP esperando conexiones de clientes**.

d. Obtener sólo los servicios UDP que están esperando comunicaciones.

```
redes@debian:~$ ss -lu
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
UNCONN	0	0	127.0.0.1:4038	0.0.0.0:*	
UNCONN	0	0	0.0.0.0:631	0.0.0.0:*	
UNCONN	0	0	0.0.0.0:33500	0.0.0.0:*	
UNCONN	0	0	0.0.0.0:mdns	0.0.0.0:*	
UNCONN	0	0	:::52048	:::*	
UNCONN	0	0	:::mdns	:::*	

El comando muestra **los servicios UDP en espera de comunicación**.

A diferencia de TCP, el estado aparece como **UNCONN** (sin conexión establecida), ya que UDP no crea sesiones.

Entre los puertos se ven mdns (multicast DNS), ipp (impresión) y otros locales, lo que indica que el sistema tiene servicios **UDP activos listos para recibir datagramas**.

e. Repetir los anteriores para visualizar el proceso del sistema asociado a la conexión.

```

redes@debian:~$ ss -ltp
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
LISTEN     0            128         0.0.0.0:ssh              0.0.0.0:*
LISTEN     0            128         127.0.0.1:ipp            0.0.0.0:*
LISTEN     0            5          127.0.0.1:4038           0.0.0.0:*
LISTEN     0            128         [::]:ssh                 [::]:*
LISTEN     0            128         [::1]:ipp                [::]:*
LISTEN     0           4096         [::1]:50051              [::]:*
LISTEN     0           4096        [::ffff:127.0.0.1]:50051  :::*

redes@debian:~$ ss -lup
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
UNCONN     0            0          127.0.0.1:4038           0.0.0.0:*
UNCONN     0            0          0.0.0.0:631              0.0.0.0:*
UNCONN     0            0          0.0.0.0:33500             0.0.0.0:*
UNCONN     0            0          0.0.0.0:mdns              0.0.0.0:*
UNCONN     0            0          [::]:52048                [::]:*
UNCONN     0            0          [::]:mdns                 [::]:*

```

El comando muestra **los servicios TCP y UDP en escucha junto con sus procesos asociados**.

En **TCP** (ss -ltp) aparecen servicios como ssh y ipp, indicando que esos procesos están activos y esperando conexiones.

En **UDP** (ss -lup), los servicios como mdns y ipp están listos para recibir datagramas, aunque UDP no mantiene conexiones activas.

En conjunto, esta salida confirma **qué procesos del sistema** están utilizando puertos TCP o UDP para ofrecer servicios en la red.

Cuadro resumen

Estado	Protocolo	Significado	Descripción breve
LISTEN	TCP	Escuchando conexiones	El servicio está esperando que un cliente inicie una conexión TCP (por ejemplo, SSH o HTTP).
ESTABLISHED	TCP	Conexión activa	Hay una sesión TCP abierta y comunicándose entre cliente y servidor.
UNCONN	UDP	Sin conexión (normal)	UDP no establece conexión; el proceso solo espera o envía datagramas.
CLOSE_WAIT, TIME_WAIT, etc.	TCP	Estados de cierre	Representan distintas etapas en el cierre de una conexión TCP.

👉 En resumen:

- TCP usa estados porque establece y controla las conexiones.
- UDP muestra UNCONN porque **no mantiene conexiones**, solo envía o recibe mensajes.

f. Obtenga la misma información planteada en los ítems anteriores usando el comando netstat.

```

redes@debian:~$ netstat -t
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State

```

netstat -t → No hay conexiones TCP activas, lo que indica que ningún proceso está comunicándose por TCP en ese momento.

```

redes@debian:~$ netstat -u
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp    0      0 192.168.64.2:bootpc     192.168.64.1:bootps     ESTABLISHED

```

netstat -u → Muestra una conexión UDP entre 192.168.64.2:bootpc y 192.168.64.1:bootps, correspondiente al servicio DHCP, usado para asignar IP automáticamente.

```
redes@debian:~$ netstat -lt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:ssh             0.0.0.0:*               LISTEN
tcp        0      0 localhost:ipp           0.0.0.0:*               LISTEN
tcp        0      0 localhost:4038          0.0.0.0:*               LISTEN
tcp6       0      0 [::]:ssh               [::]:*                  LISTEN
tcp6       0      0 localhost:ipp          [::]:*                  LISTEN
tcp6       0      0 localhost:50051        [::]:*                  LISTEN
tcp6       0      0 127.0.0.1:50051        [::]:*                  LISTEN
```

netstat -lt → Lista los servicios TCP en escucha, como ssh, ipp, 4038 y 50051, igual que con ss -lt.

```
redes@debian:~$ netstat -lu
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp        0      0 localhost:4038          0.0.0.0:*               LISTEN
udp        0      0 0.0.0.0:631            0.0.0.0:*               LISTEN
udp        0      0 0.0.0.0:33500          0.0.0.0:*               LISTEN
udp        0      0 0.0.0.0:mdns           0.0.0.0:*               LISTEN
udp6       0      0 [::]:52048             [::]:*                  LISTEN
udp6       0      0 [::]:mdns              [::]:*                  LISTEN
```

netstat -lu → Muestra los servicios UDP en espera, como mdns, ipp y algunos puertos locales.

```
redes@debian:~$ netstat -ltp
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:ssh             0.0.0.0:*               LISTEN      -
tcp        0      0 localhost:ipp           0.0.0.0:*               LISTEN      -
tcp        0      0 localhost:4038          0.0.0.0:*               LISTEN      -
tcp6       0      0 [::]:ssh               [::]:*                  LISTEN      -
tcp6       0      0 localhost:ipp          [::]:*                  LISTEN      -
tcp6       0      0 localhost:50051        [::]:*                  LISTEN      -
tcp6       0      0 127.0.0.1:50051        [::]:*                  LISTEN      -

redes@debian:~$ netstat -lup
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
udp        0      0 localhost:4038          0.0.0.0:*               LISTEN      -
udp        0      0 0.0.0.0:631            0.0.0.0:*               LISTEN      -
udp        0      0 0.0.0.0:33500          0.0.0.0:*               LISTEN      -
udp        0      0 0.0.0.0:mdns           0.0.0.0:*               LISTEN      -
udp6       0      0 [::]:52048             [::]:*                  LISTEN      -
udp6       0      0 [::]:mdns              [::]:*                  LISTEN      -
```

netstat -ltp y netstat -lup → Añaden la columna del proceso asociado (PID/Program name), aunque algunos aparecen como “-” porque se necesita ejecutar con permisos de superusuario (sudo) para ver todos.

Cuadro resumen

Comando	Qué muestra	Resultado principal
netstat -t	Conexiones TCP activas	Ninguna conexión establecida
netstat -u	Conexiones UDP activas	Comunicación DHCP (bootpc ↔ bootps)
netstat -lt	Servicios TCP en escucha	ssh, ipp, 4038, 50051
netstat -lu	Servicios UDP en escucha	mdns, ipp, 33500, etc.
netstat -ltp / -lup	Igual que los anteriores, pero con proceso	Algunos procesos no visibles sin permisos root
En conclusión, los resultados son coherentes con los obtenidos por ss, confirmando qué servicios están en escucha y qué protocolos están en uso en la máquina.		

Cuadro comparativo entre ss y netstat

Aspecto	ss	netstat
Función principal	Muestra conexiones y sockets activos del sistema.	Muestra conexiones de red y estadísticas de protocolos.
Velocidad y precisión	Más rápido y moderno (usa datos del kernel en tiempo real).	Más lento, depende de herramientas antiguas (/proc/net).
Disponibilidad	Incluido por defecto en la mayoría de las distribuciones modernas (reemplazo oficial de netstat).	Puede requerir instalar net-tools.
Sintaxis común	ss -tulnp	netstat -tulnp
Columnas principales	Estado, dirección local/remota, puerto, proceso.	Protocolo, colas (Recv-Q / Send-Q), direcciones, estado.
Soporte IPv6	Completo y automático.	Limitado, necesita opciones extra (-6).
Permisos para ver procesos	-p muestra PID y programa (requiere permisos root).	Igual, con -p (también puede requerir root).
Estado "UNCONN"	Muestra sockets UDP sin conexión activa.	Muestra UDP sin estado "establecido", sin usar "UNCONN".
Uso recomendado	Diagnóstico moderno, scripts y análisis en vivo.	Compatibilidad y comparación con herramientas antiguas.

👉 En resumen:
ss es la versión moderna, más rápida y precisa de netstat, aunque ambos sirven para observar puertos, conexiones y servicios activos.

10. ¿Qué sucede si llega un segmento TCP con el flag SYN activo a un host que no tiene ningún proceso esperando en el puerto destino de dicho segmento (es decir, el puerto destino no está en estado LISTEN)?

✓ Notas rápidas:

- -S → activa el flag SYN.
- -p → indica el puerto de destino.
- 127.0.0.1 → apunta a tu propia máquina (localhost).
- Usá sudo porque hping3 necesita permisos administrativos para enviar paquetes TCP manualmente.

- -c 1 → envía un solo paquete y termina.

a. Utilice hping3 para enviar paquetes TCP al puerto destino 22 de la máquina virtual con el flag SYN activado.

```
redes@debian:~$ sudo hping3 -S -p 22 -c 1 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): S set, 40 headers + 0 data bytes
len=44 ip=127.0.0.1 ttl=64 DF id=0 sport=22 flags=SA seq=0 win=65495 rtt=6.8 ms

--- 127.0.0.1 hping statistic ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 6.8/6.8/6.8 ms
```

El resultado muestra el flag SA (SYN-ACK), lo que significa que el puerto 22 está abierto y en estado LISTEN.

El host respondió al SYN con un ACK, indicando que el servicio (SSH) está disponible y listo para iniciar una conexión TCP.

b. Utilice hping3 para enviar paquetes TCP al puerto destino 40 de la máquina virtual con el flag SYN activado.

```
redes@debian:~$ sudo hping3 -S -p 40 -c 1 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): S set, 40 headers + 0 data bytes
len=40 ip=127.0.0.1 ttl=64 DF id=0 sport=40 flags=RA seq=0 win=0 rtt=7.0 ms

--- 127.0.0.1 hping statistic ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 7.0/7.0/7.0 ms
```

En este caso, la respuesta tiene el flag RA (RST-ACK).

Eso significa que no hay ningún proceso escuchando en ese puerto, por lo tanto, el host rechaza la conexión enviando un paquete de reinicio (RST).

c. ¿Qué diferencias nota en las respuestas obtenidas en los dos casos anteriores? ¿Puede explicar a qué se debe? (Ayuda: utilice el comando ss visto anteriormente).

```
redes@debian:~$ ss -lt
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0 128 0.0.0.0:ssh 0.0.0.0:*
LISTEN 0 128 127.0.0.1:ipp 0.0.0.0:*
LISTEN 0 5 127.0.0.1:4038 0.0.0.0:*
LISTEN 0 128 [::]:ssh [::]:*
LISTEN 0 128 [::]:ipp [::]:*
LISTEN 0 4096 [::]:50051 [::]:*
LISTEN 0 4096 [::ffff:127.0.0.1]:50051 *:*
```

Con ss -lt se ve que el puerto 22 (ssh) aparece en estado LISTEN, mientras que el puerto 40 no figura, confirmando que no hay servicio esperando en ese puerto.

Cuadro resumen

Caso	Puerto	Respuesta (flags)	Estado del puerto	Significado
a	22	SA (SYN-ACK)	LISTEN	El servicio SSH está activo y acepta conexiones.
b	40	RA (RST-ACK)	Cerrado	No hay servicio escuchando; el host rechaza el intento.

👉 En resumen:

Cuando un SYN llega a un puerto abierto, el host responde SYN-ACK (inicio normal de TCP).

Si el puerto está cerrado (sin proceso en LISTEN), responde RST-ACK, indicando que no puede establecer la conexión.

11. ¿Qué sucede si llega un datagrama UDP a un host que no tiene ningún proceso esperando en el puerto destino de dicho datagrama (es decir, que dicho puerto no está en estado LISTEN)?

✅ Notas útiles:

- --udp → usa el protocolo UDP.
- -p → puerto destino.
- -c 1 → envía un solo datagrama.
- 127.0.0.1 → dirección local (localhost).
- Ejecuté con sudo porque hping3 necesita permisos para enviar paquetes manualmente.

a. Utilice hping3 para enviar datagramas UDP al puerto destino 5353 de la máquina virtual.

```
redes@debian:~$ sudo hping3 --udp -p 5353 -c 1 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): udp mode set, 28 headers + 0 data bytes

--- 127.0.0.1 hping statistic ---
1 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

El resultado muestra "0 packets received", pero sin error ICMP, lo que significa que el datagrama fue enviado correctamente.

El puerto 5353 (mDNS) está en uso —como se ve con ss -lu—, por eso el host acepta el datagrama aunque no envíe respuesta (UDP no garantiza respuesta automática).

b. Utilice hping3 para enviar datagramas UDP al puerto destino 40 de la máquina virtual.

```
redes@debian:~$ sudo hping3 --udp -p 40 -c 1 127.0.0.1
HPING 127.0.0.1 (lo 127.0.0.1): udp mode set, 28 headers + 0 data bytes
ICMP Port Unreachable from ip=127.0.0.1 name=localhost
status=0 port=2119 seq=0

--- 127.0.0.1 hping statistic ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 11.5/11.5/11.5 ms
```

En este caso aparece el mensaje: ICMP Port Unreachable

Esto indica que no hay ningún proceso escuchando en ese puerto, por lo tanto, el host devuelve un mensaje ICMP informando que el puerto de destino no está disponible.

c. ¿Qué diferencias nota en las respuestas obtenidas en los dos casos anteriores? ¿Puede explicar a qué se debe? (Ayuda: utilice el comando ss visto anteriormente).

```
redes@debian:~$ ss -lu
State      Recv-Q    Send-Q      Local Address:Port      Peer Address:Port      Process
UNCONN     0          0           127.0.0.1:4038          0.0.0.0:*
UNCONN     0          0           0.0.0.0:631             0.0.0.0:*
UNCONN     0          0           0.0.0.0:33500           0.0.0.0:*
UNCONN     0          0           0.0.0.0:mdns             0.0.0.0:*
UNCONN     0          0           [::]:52048              [::]:*
UNCONN     0          0           [::]:mdns                [::]:*
```

En la salida se observa que el puerto 5353 (mDNS) figura en estado UNCONN, lo que confirma que el servicio está disponible, mientras que el 40 no aparece porque no hay ningún proceso esperando.

Cuadro resumen

Caso	Puerto	Resultado	Estado del puerto	Explicación
a	5353	0 packets received (sin error)	Abierto (mDNS activo)	El datagrama llega a un servicio válido, aunque UDP no exige respuesta.
b	40	ICMP Port Unreachable	Cerrado	No hay servicio; el host responde con ICMP para indicar el error.

👉 En resumen:

Cuando un datagrama UDP llega a un puerto abierto, simplemente se entrega al proceso (sin confirmación).

Si llega a un puerto cerrado, el host responde con un mensaje ICMP "Port Unreachable".

12. Investigue los distintos tipos de estado que puede tener una conexión TCP .

🌱 Estados de una conexión TCP		
Estado TCP	Descripción resumida	Cuándo ocurre
CLOSED	No hay conexión activa. Es el estado inicial y final.	Antes de iniciar o después de cerrar una conexión.
LISTEN	El servidor está esperando solicitudes (SYN) de clientes.	Un proceso servidor está "escuchando" en un puerto.
SYN-SENT	Se envió un paquete SYN y se espera el SYN-ACK del otro extremo.	Cliente inicia la conexión.
SYN-RECEIVED	Se recibió un SYN y se respondió con SYN-ACK, esperando el ACK final.	Servidor durante el saludo de tres vías.
ESTABLISHED	Conexión abierta y activa, lista para enviar/recibir datos.	Luego del <i>three-way handshake</i> .
FIN-WAIT-1	Se envió un FIN para cerrar la conexión, esperando el ACK.	Un extremo inicia el cierre.
FIN-WAIT-2	Se recibió el ACK del FIN y se espera el FIN del otro extremo.	Cierre parcial de la conexión.
CLOSE-WAIT	Se recibió un FIN del otro lado y se espera cerrar la propia parte.	El otro extremo pidió cerrar primero.
CLOSING	Ambos extremos enviaron FIN casi al mismo tiempo.	Cierre simultáneo.
LAST-ACK	Se envió FIN después de recibir FIN, esperando el ACK final.	Fase final del cierre.
TIME-WAIT	Conexión cerrada, pero se espera un tiempo para asegurar que no queden paquetes en la red.	Luego de completar el cierre (espera 2×MSL).

🔄 Flujo típico resumido	
Cliente: CLOSED → SYN-SENT → ESTABLISHED → FIN-WAIT-1 → FIN-WAIT-2 → TIME-WAIT → CLOSED Servidor: LISTEN → SYN-RECEIVED → ESTABLISHED → CLOSE-WAIT → LAST-ACK → CLOSED	Copy

En resumen: **TCP** usa estos **estados** para **controlar cada fase de la conexión**, desde el **handshake** inicial hasta el **cierre ordenado**, garantizando que todos los datos se transmitan correctamente.

13. Dada la siguiente salida del comando ss, responda:

```

Netid State  Recv-Q Send-Q Local Address:Port      Peer Address:Port    Process
tcp    LISTEN   0      128      *:22                  *:.*                  users:((("sshd",pid=468,fd=29))
tcp    LISTEN   0      128      *:80                  *:.*                  users:((("apache2",pid=991,fd=95))
udp    LISTEN   0      128      163.10.5.222:53      *:.*                  users:((("named",pid=452,fd=10))
tcp    ESTAB    0      0      163.10.5.222:59736    64.233.163.120:443    users:((("x-www-browser",pid=1079,fd=51))
tcp    CLOSE-WAI  T 0      0      163.10.5.222:41654    200.115.89.30:443
users:((("x-www-browser",pid=1079,fd=50))
tcp    ESTAB    0      0      163.10.5.222:59737    64.233.163.120:443    users:((("x-www-browser",pid=1079,fd=55))
tcp    ESTAB    0      0      163.10.5.222:33583    200.115.89.15:443     users:((("x-www-browser",pid=1079,fd=53))
tcp    ESTAB    0      0      163.10.5.222:45293    64.233.190.99:443     users:((("x-www-browser",pid=1079,fd=59))
tcp    LISTEN   0      128      *:25                  *:.*                  users:((("postfix",pid=627,fd=3))
tcp    ESTAB    0      0      127.0.0.1:22          127.0.0.1:41220       users:((("sshd",pid=1418,fd=3),
                                ("sshd",pid=1416,fd=3))
tcp    ESTAB    0      0      163.10.5.222:52952    64.233.190.94:443     users:((("x-www-browser",pid=1079,fd=29))
tcp    TIME-WAIT 0      0      163.10.5.222:36676    54.149.207.17:443     users:((("x-www-browser",pid=1079,fd=3))
tcp    ESTAB    0      0      163.10.5.222:52960    64.233.190.94:443     users:((("x-www-browser",pid=1079,fd=67))
tcp    ESTAB    0      0      163.10.5.222:50521    200.115.89.57:443     users:((("x-www-browser",pid=1079,fd=69))
tcp    SYN-SENT 0      0      163.10.5.222:52132    43.232.2.2:9500       users:((("x-www-browser",pid=1079,fd=70))
tcp    ESTAB    0      0      127.0.0.1:41220       127.0.0.1:22          users:((("ssh",pid=1415,fd=3))
udp    LISTEN   0      128      127.0.0.1:53          *:.*                  users:((("named",pid=452,fd=9))

```

a. ¿Cuántas conexiones hay establecidas?

Hay **9 conexiones TCP en estado ESTAB (establecidas)**.

Indican comunicaciones activas (por ejemplo, el navegador x-www-browser con distintos servidores remotos y procesos SSH locales).

b. ¿Cuántos puertos hay abiertos a la espera de posibles nuevas conexiones?

Se observan **4 puertos en estado LISTEN**:

- TCP 22 → sshd (espera conexiones SSH)
- TCP 80 → apache2 (servidor web HTTP)
- TCP 25 → postfix (servidor de correo)
- UDP 53 → named (servidor DNS)

c. El cliente y el servidor de las comunicaciones HTTPS (puerto 443), ¿residen en la misma máquina?

No. El puerto local es **aleatorio** (59736, 59737, 52952, etc.) y el remoto 443, lo que indica que el navegador (x-www-browser) actúa como cliente y los servidores HTTPS son externos.

d. El cliente y el servidor de la comunicación SSH (puerto 22), ¿residen en la misma máquina?

Sí. Hay una conexión entre 127.0.0.1:xxxx y 127.0.0.1:22, ambas direcciones locales, lo que indica una conexión SSH local (loopback).

```
tcp ESTAB 0 0 127.0.0.1:41220 127.0.0.1:22 users:(("ssh",pid=1415,fd=3))
tcp ESTAB 0 0 127.0.0.1:22 127.0.0.1:41220 users:(("sshd",pid=1418,fd=3))
```

e. Liste los nombres de todos los procesos asociados con cada comunicación. Indique para cada uno si se trata de un proceso cliente o uno servidor.

En la columna "Process" aparecen los programas (entre paréntesis) que están usando los puertos, por ejemplo:

```
users:(("sshd",pid=468,fd=29))
users:(("apache2",pid=991,fd=95))
users:(("named",pid=452,fd=10))
users:(("postfix",pid=627,fd=3))
users:(("x-www-browser",pid=1079,fd=51))
users:(("ssh",pid=1415,fd=3))
```

Cada uno representa un proceso activo y está asociado a una conexión o a un puerto en escucha.

Cómo darse cuenta?

- ◆ Puertos bajos (≤ 1024) → suelen ser de servidor (ej: 22 SSH, 25 SMTP, 53 DNS, 80 HTTP, 443 HTTPS).
- ◆ Puertos altos (> 1024) → usados por clientes para iniciar conexiones.
- ◆ Si el puerto está en LISTEN, es servidor; si está en ESTAB, mirá el puerto:
 - si es alto → cliente,
 - si es bajo → servidor.

Respuesta

Proceso	Protocolo / Puerto	Rol	Descripción breve
sshd	TCP / 22	Servidor	Servicio SSH que escucha conexiones entrantes.
ssh	TCP / puerto alto → 22	Cliente	Inicia conexión SSH hacia el servidor (local o remoto).
apache2	TCP / 80	Servidor	Atiende solicitudes HTTP (web).
x-www-browser	TCP / puerto alto → 443	Cliente	Navegador que se conecta a servidores HTTPS.
named	UDP / 53	Servidor	Servicio DNS que resuelve nombres.
postfix	TCP / 25	Servidor	Servicio de correo (SMTP).

Conclusión: Los procesos con puertos bien conocidos (22, 25, 53, 80) actúan como servidores, mientras que los que usan puertos altos o conexiones salientes actúan como clientes (por ejemplo ssh y x-www-browser).

f. ¿Cuáles conexiones tuvieron el cierre iniciado por el host local y cuáles por el remoto?

- Local → estado CLOSE-WAIT: el host local recibió FIN del remoto.
- Remoto → estado TIME-WAIT: el host local envió FIN y espera confirmación.

En la salida, se ve 1 conexión CLOSE-WAIT y 1 conexión TIME-WAIT.

g. ¿Cuántas conexiones están aún pendientes por establecerse?

Hay 1 conexión en estado SYN-SENT, lo que indica que el cliente envió un SYN pero todavía no recibió el SYN-ACK.

14. Dadas las salidas de los siguientes comandos ejecutados en el cliente y el servidor, responder:

```
servidor# ss -natu | grep 110
tcp    LISTEN 0 0          *:110      *: *
tcp    SYN-RCV 0 0          157.0.0.1:110 157.0.11.1:52843

cliente# ss -natu | grep 110
tcp    SYN-SENT 0 1          157.0.11.1:52843 157.0.0.1:110
```

a. ¿Qué segmentos llegaron y cuáles se están perdiendo en la red?*Para entender*

Cuando un cliente quiere abrir una conexión TCP con un servidor, se hace el "saludo de tres vías" (3-way handshake). Los pasos normales son:

- 1 Cliente → Servidor: envía un segmento con el flag **SYN** (quiere iniciar la conexión).
 - 2 Servidor → Cliente: responde con un **SYN+ACK** (acepta y confirma).
 - 3 Cliente → Servidor: manda un **ACK** (confirma la respuesta).
- Ambos pasan al estado **ESTABLISHED** (conexión abierta).

- El cliente está en estado **SYN-SENT** → ya envió el segmento **SYN** al servidor.
 - El servidor está en estado **SYN-RECV** → recibió el **SYN** y envió el **SYN+ACK** de respuesta.
 - Como el cliente no pasa a **ESTAB**, significa que el **ACK** final del cliente no llegó al servidor.
- El segmento perdido es el **ACK** del cliente (tercer paso del saludo de 3 vías).

Si la conexión se hubiese completado correctamente

```
Salida esperada si el ACK llega correctamente (conexión TCP establecida)

bash
# En el servidor
servidor# ss -natu | grep 110
tcp LISTEN  0 0  *:110      **
tcp ESTAB   0 0  157.0.0.1:110  157.0.11.1:52843

# En el cliente
cliente# ss -natu | grep 110
tcp ESTAB   0 0  157.0.11.1:52843  157.0.0.1:110
```

El cliente y el servidor completaron correctamente el saludo de tres vías (**SYN** → **SYN+ACK** → **ACK**).

Por eso ambos aparecen en estado **ESTABLISHED**, indicando que la conexión TCP se estableció con éxito y ya pueden intercambiar datos.

b. ¿A qué protocolo de capa de aplicación y de transporte se está intentando conectar el cliente?

- Capa de transporte: **TCP** (se observa por los estados **SYN-SENT** y **SYN-RECV**).
- Capa de aplicación: **POP3**, porque el puerto 110 es el puerto estándar de ese protocolo.

Entonces el cliente intenta abrir una conexión TCP para un servicio **POP3**.

c. ¿Qué flags tendría seteado el segmento perdido?

El segmento perdido corresponde al último paso del saludo de tres vías (el **ACK** final que envía el cliente).

Por eso, solo lleva el flag **ACK** activado (**ACK = 1**) para confirmar la recepción del **SYN+ACK** del servidor.

El flag **SYN** está desactivado (**SYN = 0**), ya que la conexión ya fue iniciada.

En resumen: el segmento perdido es un paquete TCP con flag **ACK**.

15.

Ejercicio 15 — CORE y análisis de conexiones TCP		
Inciso	Acción realizada / comando	Resultado / Explicación
a.	En ambos equipos se ejecutó: <code>watch -n1 'ss -nat'</code>	Permite observar en tiempo real los cambios de estado de las conexiones TCP.
b.	En Servidor: <code>ncat -lk 8001</code>	El puerto 8001/TCP queda en estado LISTEN , esperando conexiones entrantes persistentes.
c.	En Cliente1: <code>ncat 10.0.0.20 8001</code>	Se establece una conexión TCP entre Cliente1 (puerto origen aleatorio) y Servidor (puerto destino 8001). En <code>ss</code> aparece ESTAB en ambos nodos.
d.	Se abre otra conexión igual desde Cliente1	Aparece otra conexión con distinto puerto origen → cada conexión TCP se identifica por el cuádruple: IP origen , IP destino , puerto origen y puerto destino .
e.	Se prueban varias conexiones hacia el mismo puerto destino (8001)	Es posible tener múltiples conexiones simultáneas, porque cada una tiene un puerto origen distinto. Los datos no se mezclan gracias a este identificador único.
f.i.	Cierre desde Cliente (Ctrl+C en ncat del cliente)	En Cliente: conexión pasa a FIN-WAIT-1 / FIN-WAIT-2 . En Servidor: CLOSE-WAIT hasta que finaliza completamente.
f.ii.	Cierre del servicio en Servidor (Ctrl+C en ncat del servidor)	Todas las conexiones pasan a CLOSED o TIME-WAIT , según el momento del cierre.
f.iii.	Cierre manual desde Cliente sin servicio activo	El cliente detecta que no hay respuesta (RST recibido). Conexión termina en CLOSED .

Resumen

Resumen final
• El puerto 8001 en el servidor actúa como punto de escucha (LISTEN). • Cada conexión TCP se diferencia por su puerto origen. • TCP garantiza que los datos de distintas conexiones no se mezclen. • Los estados observados (LISTEN, ESTAB, CLOSE-WAIT, TIME-WAIT) representan las distintas etapas del ciclo de vida de una conexión.